

Markdown to PDF

We've converted 2,429,407 Markdown files to PDF and counting!

To convert your Markdown to PDF simply start by typing in the editor or pasting from your clipboard.

If y# Claude Code with CCS (Claude Code Switch) for Profile Switching

Quick Start

```
# Install CCS
npm install -g @kaitranntt/ccs

# Create profile: ~/.ccs/onprem.settings.json
cat > ~/.ccs/onprem.settings.json << 'EOF'
{
  "env": {
    "CLAUDE_CODE_USE_FOUNDRY": "0",
    "CLAUDE_CODE_USE_BEDROCK": "0",
    "CLAUDE_CODE_USE_VERTEX": "0",
    "ANTHROPIC_BASE_URL": "https://llm-api2.bunker.onprem.ai",
    "ANTHROPIC_AUTH_TOKEN": "sk-rV9vrhHWjYc0Waj6y3FSww",
    "ANTHROPIC_MODEL": "qwen3-coder-next-q8"
  }
}
EOF

# Register profile
ccs api discover --register

# Use it
ccs onprem
```

Status

Deprecated - This approach should not be used for new setups. See the deprecation notice below.

Deprecation Notice

CCS (Claude Code Switch) is Deprecated

The CCS (Claude Code Switch) approach documented here is **deprecated** and should not be used for new setups.

Why deprecated:

- CCS adds an unnecessary dependency layer (@kaitranntt/ccs)
- The profile isolation problem has simpler solutions (see ADR 0001 direct setup)
- Maintenance burden of an external switch tool outweighs benefits

Recommended alternatives:

1. **Direct setup** - See [ADR 0001](#) for direct configuration without CCS
2. **Proxy setup** - See [ADR 0002](#) for the claude-code-proxy approach if you need an OpenAI-to-Anthropic translation layer

Migration: If you're currently using CCS, migrate to the direct setup based on your needs. The direct setup uses `~/.claude/settings.json` with Bunker config and avoids the external switch tool.

Context

Running Claude Code with multiple configurations (e.g., corporate Foundry/Azure AND onprem Bunker) is problematic because:

1. Claude Code's default config (`~/.claude/settings.json`) **always loads and overrides** custom configs
2. `CLAUDE_CONFIG_DIR` does **not** fully isolate configurations
3. Environment variables like `ANTHROPIC_BASE_URL` are **ignored** when Foundry mode is enabled

See [ADR 0001](#) for details on the config pollution problem.

CCS (Claude Code Switch) solves this by providing proper profile isolation and a simple CLI for switching between configurations.

Installation

```
npm install -g @kaitranntt/ccs
```

Alternative package managers:

```
pnpm add -g @kaitranntt/ccs # 70% less disk space
bun add -g @kaitranntt/ccs # faster
```

Setup: Create Onprem Profile

Step 1: Create the settings file

Create `~/.ccs/onprem.settings.json`:

```
{
  "env": {
    "CLAUDE_CODE_USE_FOUNDRY": "0",
    "CLAUDE_CODE_USE_BEDROCK": "0",
    "CLAUDE_CODE_USE_VERTEX": "0",
    "ANTHROPIC_BASE_URL": "https://llm-api2.bunker.onprem.ai",
    "ANTHROPIC_AUTH_TOKEN": "<YOUR_API_KEY>",
    "ANTHROPIC_MODEL": "qwen3-coder-next-q8",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "qwen3-coder-next-q8",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "qwen3-coder-next-q8",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "qwen3-coder-next-q8"
  }
}
```

Critical settings:

- `CLAUDE_CODE_USE_FOUNDRY=0` - Disables Microsoft Azure/Foundry
- `CLAUDE_CODE_USE_BEDROCK=0` - Disables Amazon Bedrock
- `CLAUDE_CODE_USE_VERTEX=0` - Disables Google Vertex AI
- `ANTHROPIC_AUTH_TOKEN` - Use this, not `ANTHROPIC_API_KEY`. CCS uses the auth token format.

Note: Setting all three cloud provider flags to `0` ensures your onprem endpoint is used regardless of what's configured in the user's default `~/.claude/settings.json`.

Step 2: Register the profile

```
# Discover and register the settings file
ccs api discover --register
```

Step 3: Verify

```
# List registered profiles
ccs api list

# Check environment variables
ccs env onprem
```

Usage

Run Claude Code with onprem profile

```
ccs onprem
```

This launches Claude Code with all the environment variables from your onprem settings file.

Run with a prompt directly

```
ccs onprem "explain this code"
```

Check profile status

```
ccs api list      # List all API profiles
ccs env onprem    # Show env vars for profile
ccs doctor        # Run diagnostics
```

How CCS Solves the Config Pollution Problem

CCS sets environment variables that **override** values from `~/.claude/settings.json` :

- 1. **Env vars take precedence** - CCS injects env vars before launching Claude Code, and env vars override settings.json values
- 2. **Explicit Foundry disable** - Setting `CLAUDE_CODE_USE_FOUNDRY=0` in your CCS profile overrides `CLAUDE_CODE_USE_FOUNDRY=1` from the default config
- 3. **No config file modification** - Unlike `ccs persist`, running `ccs <profile>` never touches `~/.claude/settings.json`

Key insight: The solution isn't to hide the default config, but to explicitly disable Foundry mode via env var. This allows both configs to coexist:

- `claude` → uses default config with Foundry/Azure
- `ccs onprem` → uses onprem endpoint (Foundry disabled via env var)

WebSearch Fallback

When using third-party providers (like Bunker), you'll see:

```
[X] WebSearch: Install: npm i -g @google/gemini-cli
[i] WebSearch: No CLI tools installed
```

This is expected. Third-party providers don't have access to Anthropic's native WebSearch tool. CCS can provide fallback web search via external CLI tools.

Options

Tool	Cost	Requires	Install
Gemini CLI	Free (1000 req/day)	Google account	<code>npm i -g @google/gemini-cli</code>
OpenCode	Free tier available	OpenCode Zen signup	<code>curl -fsSL https://opencode.ai/install bash</code>
Grok CLI	Paid	GROK_API_KEY	<code>npm i -g @vibe-kit/grok-cli</code>

No account-free option exists for web search fallback.

Recommended: Gemini CLI

Most users already have a Google account:

```
npm i -g @google/gemini-cli
gemini # opens browser for Google OAuth (one-time)
```

After authentication, CCS automatically uses Gemini CLI for web searches on non-Anthropic endpoints.

Skip Web Search

If you don't need web search in onprem sessions, just ignore the warning. Claude Code works fine without it - you just won't have access to the WebSearch tool.

Multiple Profiles

You can create multiple profiles for different environments:

```
# Create settings files
~/.ccs/work.settings.json      # Corporate Foundry
~/.ccs/onprem.settings.json    # Bunker onprem
~/.ccs/personal.settings.json  # Personal API key

# Register all
ccs api discover --register

# Use any profile
ccs work "task"
ccs onprem "task"
ccs personal "task"
```

Troubleshooting

Profile not found after creating settings file

Run `ccs api discover --register` to register orphan settings files.

"Profile is type 'undefined'" error

The settings file format is incorrect. Ensure it has the `env` object with `ANTHROPIC_AUTH_TOKEN` (not `ANTHROPIC_API_KEY`).

Still connecting to wrong endpoint

Use `ccs env <profile>` to verify the environment variables are correct. If using mitmproxy, check traffic is going to the expected endpoint.

Troubleshooting

If CCS is not working for your setup, see [ADR 0001: Claude Code with Bunker onprem \(direct\)](#) for alternative approaches including:

- Direct configuration without CCS
- Wrapper scripts for config isolation
- Manual environment variable setup

References

- [CCS GitHub](#) - 1.1k+ stars, MIT license
- [CCS Website](#)
- [ADR 0001: Claude Code with Bunker onprem \(direct\)](#) - Documents the config pollution problem and alternative solutions our Markdown is in a file clear this content and drop your file into this editor.

tip: click on the pencil icon on the left to clear the editor)

GitHub flavoured styling by default

We now use GitHub flavoured styling by default.